LAB – 2 Single-Cycle RISC Processor

Prepared by

Student name: Hadi Beiram Student ID: 300299908

Student name Hao Sun Student ID: 300226096

Group #: A0X- 3

CEG 3156: Computer Systems Design (Winter 2026)

Prof. Rami Abielmona

Presented to: Youssef Elfaramawy & Ro'yah Radaideh

TA and Lab Supervisor:

uOttawa

School of Electrical Engineering and Computer Science

École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie

University of Ottawa | Université d'Ottawa

Submission Date: March 18, 2026

Table of content

LAB – 2 Single-Cycle RISC Processor 1
1. Introduction 3
1.1 Summary 3
1.2 Background Information 3
1.2.1 Single-cycle RISC processor concept 3
1.2.2 Supported instruction types 3
1.2.3 Why a single-cycle datapath is used 4
1.2.4 Pre-lab: supporting the bne instruction 4
2. Design 4
2.1 Design methodology 4
2.2 Top-level integration 4
2.3 Main datapath implementation 5
2.3.1 Program counter and instruction fetch 5
2.3.2 Register file 5
2.3.3 Sign extension and branch address generation 5
2.3.4 ALU and ALU control 5
2.3.5 Data memory 5
2.3.6 Multiplexers and write-back path 5
2.3.7 Output monitoring logic 6
2.4 Main control unit 6
2.5 Pre-lab modification for bne 6
2.6 Benchmark program and memory initialization 7
3. Simulation and Results 7
3.1 Functional verification 7
3.1.1 Load and Store Instructions 8
3.1.2 R-Type Instructions 8
3.1.3 Jump and Branch Instructions 8
3.2 Timing Discussion 8
3.3 Maximum Clock Frequency 8
3.4 CPU Execution Time 8
4. Design Obstacles and Solutions 8
5. Conclusion 9

6. Appendices 9
5.2 Lab Book (CEG 3156 Lab 2) 9
5.3 Lab Code Zip 9
5.4 Graphs / Waveforms / Screenshots 9
5.5 GitHub https://github.com/xuanwu12345/ceg-3156-group-a0x-3.git 12

1. Introduction

1.1 Summary

In this laboratory, we designed and implemented a single-cycle RISC processor in VHDL using a structural RTL methodology. The design follows the MIPS-style single-cycle datapath discussed in the lab manual, but it is adapted to the laboratory constraints of 8-bit datapaths, 32-bit instructions, 8-bit data memory, and monitored output signals for board-level demonstration. The processor supports memory-reference instructions (lw and sw), arithmetic and logic R-type instructions (add, sub, and, or, and slt), and control-flow instructions (beq and j). Our control logic also includes support for the branch-if-not-equal instruction, bne, as required in the pre-lab extension.

It first explains the processor background and instruction classes, then presents the top-level architecture and the main sub-blocks, followed by a discussion of control signals, benchmark execution, simulation intent, timing considerations, and design obstacles. The goal is to document not only what was built, but also how the design decisions map to the expected laboratory architecture.

1.2 Background Information

1.2.1 Single-cycle RISC processor concept

A single-cycle processor completes one full instruction in one clock cycle. This means instruction fetch, decode, register read, ALU execution, possible memory access, and write-back must all be completed before the next active clock edge. The main benefit of this approach is architectural simplicity, because every instruction uses a single unified datapath. The main drawback is that the clock period must be long enough to satisfy the slowest instruction path, which reduces the maximum frequency compared with pipelined or multicycle designs.

1.2.2 Supported instruction types

The implemented processor supports the core instruction groups required by the laboratory: load word (lw), store word (sw), arithmetic/logic R-type instructions, branch equal (beq), and jump (j). In the submitted

design files, the control unit also decodes branch not equal (bne), which allows the datapath to use the same comparison path as beq while changing the branch condition.

1.2.3 Why a single-cycle datapath is used

This design style is especially useful for a teaching laboratory because it makes the relationship between the instruction format, the datapath, and the control signals very explicit. Each instruction enters the processor, propagates through the same major hardware blocks, and exits in a predictable way. This makes the design easier to explain, debug, and simulate.

1.2.4 Pre-lab: supporting the bne instruction

The bne instruction reuses the same ALU comparison path as beq. The ALU subtracts the two source register values. For beq, the branch is taken when Zero = 1. For bne, the branch is taken when Zero = 0. Therefore, only a small extension is required: an additional opcode decode path and a new control signal, BranchNEQ, that participates in the next-PC selection logic.

2. Design

2.1 Design methodology

The processor was built using structural VHDL and RTL-oriented decomposition. The top level is drawn graphically, while the lower-level atomic and reusable blocks are implemented in VHDL. This matches the lab design restrictions and the same modular writing style used in the Lab 1 report. The project combines the program counter, instruction memory, register file, ALU control, ALU, sign-extension path, data memory, control unit, and the multiplexers needed to route operands, destinations, write-back data, and next-PC values.

2.2 Top-level integration

The top-level design accepts GClock, GReset, and ValueSelect[2..0] as inputs. It provides MuxOut[7..0], InstructionOut[31..0], BranchOut, ZeroOut, MemWriteOut, and RegWriteOut as outputs. Internally, the current PC addresses the instruction memory, a separate adder computes PC + 4, and the fetched instruction is split into opcode, register addresses, function bits, immediate data, and jump information. The main control unit and ALU control unit generate the selection signals that steer the datapath.

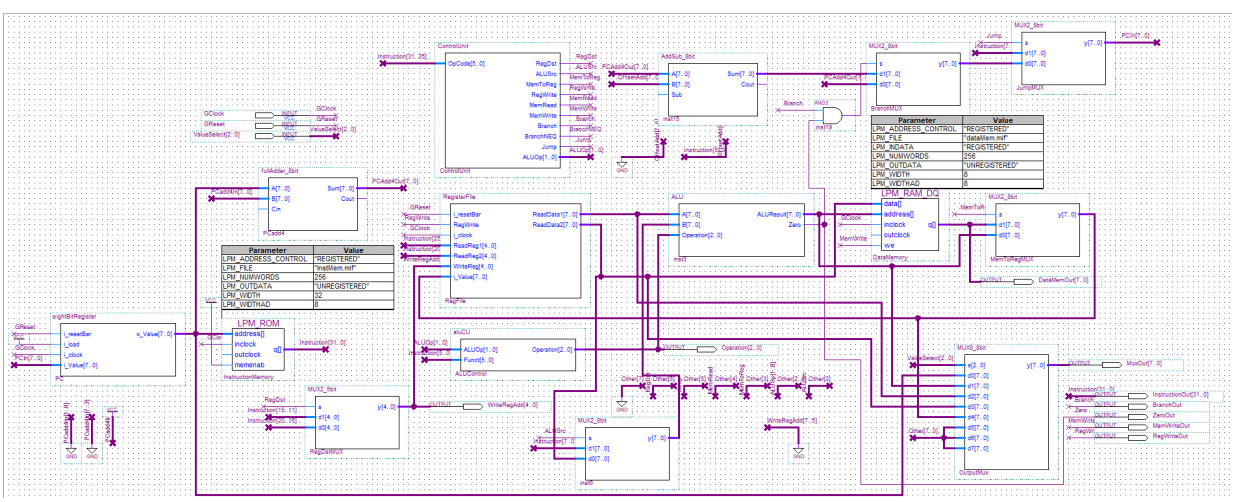


Figure 1. Single-cycle RISC processor top-level block diagram extracted from the submitted Quartus project

2.3 Main datapath implementation

2.3.1 Program counter and instruction fetch

The PC stores the address of the current instruction. In each cycle, instruction memory is read at the current PC value, and a separate adder computes PC + 4 because the instruction width is 32 bits. The next-PC logic later chooses between the sequential address, a branch target, and a jump target.

2.3.2 Register file

The submitted VHDL register file is a 32 × 8-bit structure with two read ports and one write port. Although the lab handout discusses an 8-register teaching version, the project files implement five-bit register addressing and decode the write address with a 5-to-32 decoder. This preserves the familiar MIPS field widths while still operating on 8-bit data values.

2.3.3 Sign extension and branch address generation

I-type immediates are extended before being used by the ALU or by the branch target computation. For branches, the immediate offset participates in address generation for the branch destination. For memory operations, the immediate acts as the address offset added to the base register.

2.3.4 ALU and ALU control

The ALU supports AND, OR, ADD, SUB, and SLT. The ALU control block receives ALUOp and the function field and converts them into a 3-bit operation signal. In the submitted VHDL, subtraction is implemented through the same add/sub block used by arithmetic operations, and Zero is asserted when the output bus is all zeros. This allows the same execution path to support both arithmetic instructions and branch comparisons.

```
library ieee;
use ieee.std_logic_1164.all;

entity aluCU is
   port(
      ALUOp : in std_logic_vector(1 downto 0); --Op bits
       Funct : in std_logic_vector(5 downto 0); --Function bits
       Operation : out std_logic_vector(2 downto 0)); --Out bit
end aluCU;

architecture rtl of aluCU is
SIGNAL nextOP:std_logic_vector(2 downto 0);

begin
nextOP(2) <= ALUOp(0) OR (ALUOp(1)AND Funct(1));
nextOP(1) <= NOT ALUOp(1)OR NOT Funct(2);
nextOP(0) <= ALUOp(1)AND(Funct(0)OR Funct(3));

Operation <= nextOP;
end rtl;
```

Figure 2. ALU control block used to translate ALUOp and Funct fields into the ALU operation code

2.3.5 Data memory

The processor uses separate instruction and data memories. Data memory is 8 bits wide and can be read or written depending on the control signals MemRead and MemWrite. For lw, the ALU computes the address and the memory output is routed back to the register file through the MemToReg path. For sw, the second read port of the register file provides the value written to memory.

2.3.6 Multiplexers and write-back path

Several multiplexers are used throughout the design: RegDst chooses the write register; ALUSrc chooses between register data and immediate data for the second ALU operand; MemToReg chooses between ALU output and data memory output; the next-PC multiplexers choose between PC + 4, a branch target, and a jump target; and the final output multiplexer drives MuxOut[7..0] according to ValueSelect[2..0].

2.3.7 Output monitoring logic

The final output multiplexer is especially useful for demonstration and debugging. It allows the board LEDs and simulations to show the program counter, the ALU result, the register file read ports, the write-back data, or control-related information without changing the underlying datapath.

2.4 Main control unit

The control unit decodes the 6-bit opcode and generates the major control signals: RegDst, ALUSrc, MemToReg, RegWrite, MemRead, MemWrite, Branch, BranchNEQ, Jump, and ALUOp[1..0]. In the submitted code, the decode logic explicitly recognizes r-format, lw, sw, beq, bne, and j. This is the key module that determines how each instruction uses the datapath.

```
library ieee;
use ieee.std_logic_1164.all;
--Processor Control unit

entity ControlUnit is
   port(OpCode : in std_logic_vector(5 downto 0); --OpCode
        RegDst,ALUSrc,MemToReg,RegWrite,MemRead : out std_logic;
        MemWrite,Branch,BranchNEQ : out std_logic;
        ALUOp : out std_logic_vector(1 downto 0)); --Control signals
end ControlUnit;

architecture rtl of ControlUnit is
 signal rFormat,lw,sw,beq,bne : std_logic;

begin
 --Instruction type
   rFormat <= not OpCode(6)AND not OpCode(1)AND not OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);-- 0, rFormat
   lw      <= OpCode(0)AND OpCode(1)AND not OpCode(2)AND not OpCode(3)AND not OpCode(4)AND OpCode(5);--35
   sw      <= OpCode(0)AND OpCode(1)AND not OpCode(2)AND OpCode(3)AND not OpCode(4)AND OpCode(5);--43
   beq     <= not OpCode(0)AND not OpCode(1)AND OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);
   bne     <= OpCode(0)AND not OpCode(1)AND OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);

 --Control signal outputs
   RegDst    <= rFormat;
   ALUSrc    <= lw OR sw;
   MemToReg  <= lw;
   RegWrite  <= rFormat OR lw;
   MemRead   <= lw;
   MemWrite  <= sw;
   Branch    <= beq;
   BranchNEQ <= bne;
   ALUOp(1)  <= rFormat;
   ALUOp(0)  <= beq OR bne;
 end rtl;
```

Figure 3. Main control unit block extracted from the submitted Quartus project

Instruction	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	BNE	Jump	ALUOp
R-type	1	0	0	1	0	0	0	0	0	10
lw	0	1	1	1	1	0	0	0	0	00
sw	X	1	X	0	0	1	0	0	0	00
beq	X	0	X	0	0	0	1	0	0	01
bne	X	0	X	0	0	0	0	1	0	01
j	X	X	X	0	0	0	0	0	1	XX

Table 1. Control signal summary for the main instruction classes used in the processor

2.5 Pre-lab modification for bne

To support bne, the project adds a separate decode condition in the control unit for opcode 5 and produces BranchNEQ as a dedicated signal. The ALU still performs subtraction to compare the two source registers, but the branch decision now depends on Zero = 0 instead of Zero = 1. This is an efficient extension because it reuses the existing datapath and only changes the control logic and PC selection condition.

2.6 Benchmark program and memory initialization

The benchmark memory files initialize a small instruction sequence and a small data memory image. The benchmark exercises load/store behavior, ALU operations, and control-flow behavior. The design is therefore validated at the full-processor level rather than only at the isolated component level.

Signal / File	Purpose
instMem.mif	Instruction memory image used by the 256 × 32 ROM.
dataMem.mif	Data memory image used by the 256 × 8 RAM.
ValueSelect	Chooses which internal processor value appears on MuxOut[7..0].
InstructionOut	Displays the current 32-bit instruction during simulation.
BranchOut / ZeroOut	Expose branch decision context for debugging and demonstration.

Table 2. Summary of key external observability points and memory initialization files

3. Simulation and Results

3.1 Functional verification

To verify the functionality of the single-cycle RISC processor, a top-level waveform simulation was performed. The waveform shows that the processor executes instructions in the correct sequence, including lw, R-type operations, sw, j, and branch instructions. The control signals also respond correctly: RegWriteOut is active during register write-back, MemWriteOut is active during store operations, and BranchOut changes during branch execution. In addition, Operation and ZeroOut confirm correct ALU control and comparison behavior. Overall, the waveform verifies that the processor datapath and control unit operate correctly together.

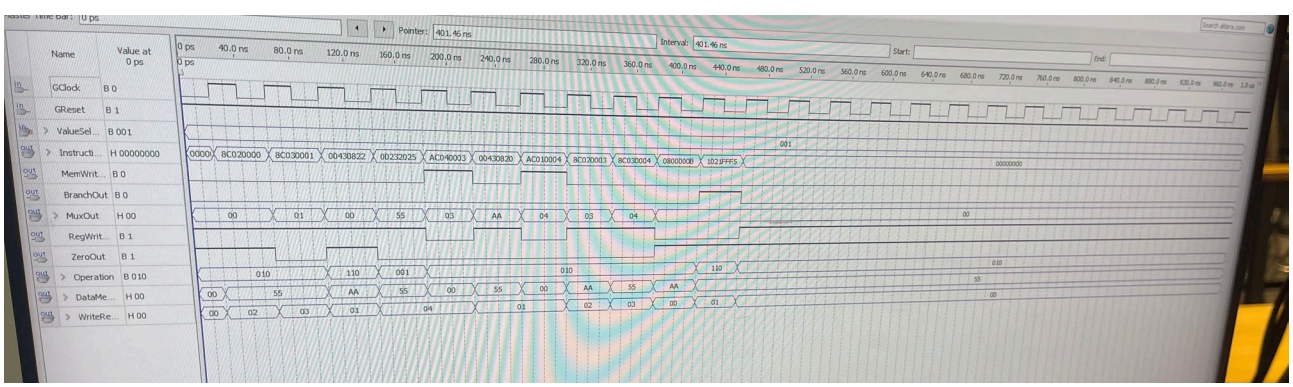


Figure 6. Overall processor timing waveform showing instruction execution, control signals, and data path outputs

3.1.1 Load and Store Instructions

The waveform confirms correct execution of lw and sw. The load instructions read values from data memory and write them back into the register file, as shown by active RegWriteOut. The store instructions activate MemWriteOut, confirming that register data is written into memory only during sw. The data values observed in the waveform are consistent with the initialized memory contents.

3.1.2 R-Type Instructions

The waveform also verifies correct execution of R-type instructions such as subtraction, OR, and addition. During these instructions, the Operation signal changes according to the required ALU function, while RegWriteOut is active to store the ALU result in the destination register. Since these instructions do not access memory, MemWriteOut remains inactive.

3.1.3 Jump and Branch Instructions

The jump and branch instructions are also correctly observed in the waveform. The jump instruction changes the instruction flow to the target address, while the branch instruction activates BranchOut and uses ZeroOut for comparison. This confirms that the next-PC selection logic works properly for control-flow instructions.

3.2 Timing Discussion

In a single-cycle processor, each instruction must finish in one clock cycle. Therefore, the clock period is determined by the slowest instruction path, which is usually the load instruction because it includes instruction fetch, register read, ALU calculation, memory access, and write-back in the same cycle.

3.3 Maximum Clock Frequency

The maximum clock frequency is limited by the longest datapath delay. In this design, the lw instruction has the slowest path, so it determines the minimum clock period. The exact maximum frequency should be confirmed using Quartus timing analysis.

3.4 CPU Execution Time

Since this is a single-cycle processor, each instruction takes one clock cycle. Therefore, CPU execution time depends on the number of executed instructions and the clock period:

$$CPU\ Execution\ Time = Instruction\ Count \times Clock\ Cycle\ Time$$

This means the processor has a CPI of 1, while the clock period is set by the slowest instruction.

4. Design Obstacles and Solutions

One challenge in this lab was connecting all processor modules correctly in one single-cycle data path. Another challenge was verifying the control signals for different instruction types. Branch and jump instructions were also more difficult because they required correct PC update logic. These issues were solved by checking the waveform simulation and debugging the signal connections step by step.

5. Conclusion

In this lab, we successfully designed and verified a single-cycle RISC processor in VHDL. The processor correctly supports memory, arithmetic, logic, branch, and jump instructions. The waveform results confirm that both the data path and control unit work properly. Overall, this lab helped us better understand processor design, instruction execution, and control signal generation.

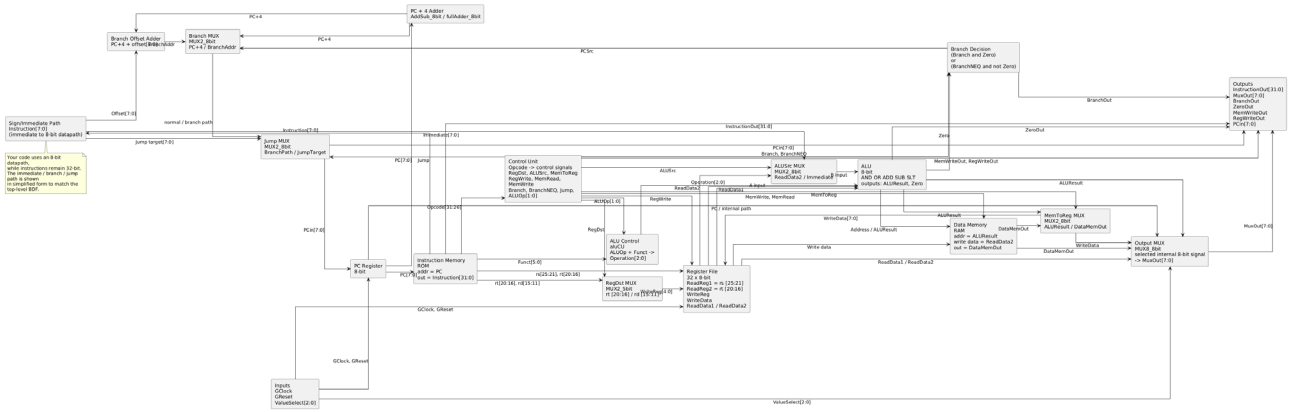
6. Appendices

5.2 Lab Book (CEG 3156 Lab 2)

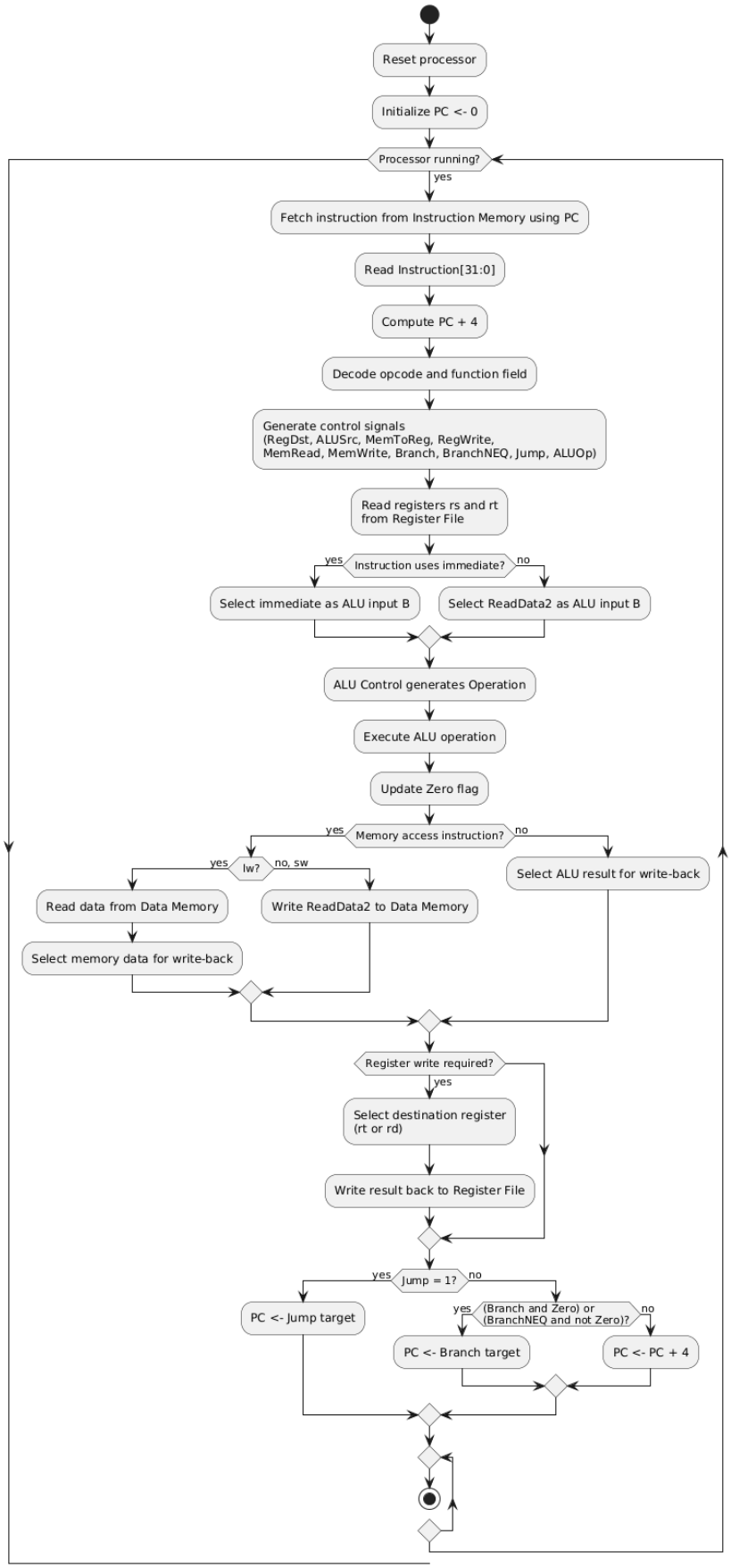
5.3 Lab Code Zip

Attach the project source files, including all VHDL files and graphical design files.

5.4 Graphs / Waveforms / Screenshots

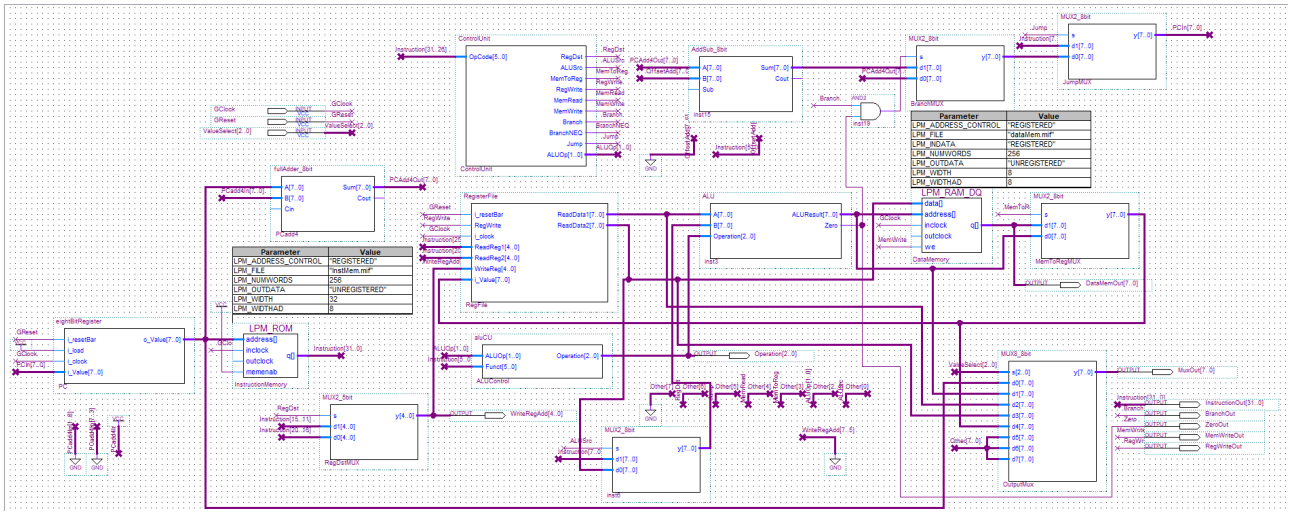


Single-Cycle RISC Processor Flowchart



```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4   entity aluCU is
5      port(
6         ALUOp : in std_logic_vector(1 downto 0); --Op bits
7            Funct : in std_logic_vector(5 downto 0); --Function bits
8            Operation : out std_logic_vector(2 downto 0)); --Out bit
9   end aluCU;
10
11  architecture rtl of aluCU is
12  SIGNAL nextOP:std_logic_vector(2 downto 0);
13
14  begin
15  nextOP(2) <= ALUOp(0) OR (ALUOp(1)AND Funct(1));
16  nextOP(1) <= NOT ALUOp(1)OR NOT Funct(2);
17  nextOP(0) <= ALUOp(1)AND(Funct(0)OR Funct(3));
18
19  Operation <= nextOP;
20  end rtl;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   --Processor Control unit
4
5   entity ControlUnit is
6      port(OpCode : in std_logic_vector(5 downto 0); --OpCode
7            RegDst,ALUSrc,MemToReg,RegWrite,MemRead : out std_logic;
8            MemWrite,Branch,BranchNEQ : out std_logic;
9            ALUOp : out std_logic_vector(1 downto 0)); --Control signals
10  end ControlUnit;
11
12  architecture rtl of ControlUnit is
13  signal rFormat,lw,sw,beq,bne : std_logic;
14
15  begin
16  --Instruction type
17     rFormat <= not OpCode(0)AND not OpCode(1)AND not OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);-- 0, rFormat
18     lw      <= OpCode(0)AND OpCode(1)AND not OpCode(2)AND not OpCode(3)AND not OpCode(4)AND OpCode(5);--35
19     sw      <= OpCode(0)AND OpCode(1)AND not OpCode(2)AND OpCode(3)AND not OpCode(4)AND OpCode(5);--43
20     beq     <= not OpCode(0)AND not OpCode(1)AND OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);
21     bne     <= OpCode(0)AND not OpCode(1)AND OpCode(2)AND not OpCode(3)AND not OpCode(4)AND not OpCode(5);
22
23  --Control signal outputs
24     RegDst    <= rFormat;
25     ALUSrc    <= lw OR sw;
26     MemToReg  <= lw;
27     RegWrite  <= rFormat OR lw;
28     MemRead   <= lw;
29     MemWrite  <= sw;
30     Branch    <= beq;
31     BranchNEQ <= bne;
32     ALUOp(1)  <= rFormat;
33     ALUOp(0)  <= beq OR bne;
34  end rtl;
```



5.5 GitHub

https://github.com/xuanwu12345/ceg-3156-group-a0x-3.git